Development and Deployment of a Full-Stack E-Commerce Application on a VPS

Members:

Marlou Apuya
Jerome Torregosa
Eumir Maling

Introduction

This project focuses on the development and deployment of a full-stack e-commerce application using a Virtual Private Server (VPS). The system was designed to simulate a real-world online store environment where users can view products, place orders, and generate sales data. The project integrates frontend development, backend API services, PostgreSQL databases, ETL automation, and analytics reporting into a single deployed system accessible through the internet.

The application uses FastAPI as the backend framework, PostgreSQL as the main transactional database, and React with Vite for the frontend interface. To support reporting and analytics, a separate analytics database was implemented together with an ETL process automated through Linux cron jobs. The entire system was deployed on a VPS using Ubuntu Linux and Nginx.

Objectives

The project aimed to:

- Build a functional full-stack e-commerce system
- Develop a backend API using FastAPI
- Store transactional data using PostgreSQL
- Create a separate analytics database for reporting
- Implement ETL automation using cron jobs
- Deploy the system publicly through a VPS
- Create a dashboard that displays analytics such as revenue, orders, and top products

System Architecture

The system follows a multi-layer architecture composed of frontend, backend, database, ETL, and analytics components.

Frontend Interface
↓
FastAPI Backend API
↓

Main Database (ecommerce_db)
↓
ETL Process (etl.py + cron)
↓
Analytics Database (analytics_db)
↓
Analytics Dashboard

The frontend communicates with the backend API to retrieve products and process orders. The backend stores transactional data inside the main PostgreSQL database. An ETL script periodically extracts and transforms the data before loading it into the analytics database used for reporting.

Frontend Development

The frontend was developed using React and Vite. The interface allows users to:

- View available products
- Place orders
- Access analytics information
- Interact with the system through a web browser

The frontend was deployed using Nginx inside the VPS to make the application publicly accessible through the internet.

Frontend URL:

http://31.97.191.52

Backend API Development

The backend was developed using FastAPI. REST API endpoints were created to manage products, users, orders, and analytics data.

Main API endpoints include:

- /products
- /orders
- /analytics

The API handles requests from the frontend and performs database operations such as inserting orders and retrieving analytics information.

API Documentation:

http://31.97.191.52:8000/docs

ETL Process

An ETL (Extract, Transform, Load) pipeline was implemented using Python.

Extract

The ETL process extracts raw transactional data from ecommerce_db, particularly from the orders and products tables.

Transform

The extracted data is processed to calculate:

- total revenue
- total orders
- top-selling products

Load

The transformed data is inserted into analytics_db for reporting purposes.

The ETL script is stored in:

etl.py

Cron Job Automation

To automate the ETL pipeline, Linux cron jobs were used. The cron scheduler automatically executes the ETL script every two minutes.

Cron configuration:

*/2 * * * * cd /root/ecommerce-project/backend && /usr/bin/python3 etl.py >> etl.log 2>&1

This automation ensures that analytics data is continuously updated without manual intervention.

VPS Deployment

The system was deployed on an Ubuntu VPS environment. Deployment involved:

- configuring Nginx
- running FastAPI with Uvicorn
- configuring PostgreSQL
- enabling external access
- connecting pgAdmin remotely

- managing Linux services and ports

The deployment made the application accessible through the public internet instead of localhost.

Analytics Dashboard

The dashboard displays processed analytics data gathered from the analytics database. Information shown includes:

- total orders
- total revenue
- top products

The dashboard demonstrates how ETL pipelines can support business reporting and decision-making.

Testing and Validation

System testing was conducted to verify:

- frontend accessibility
- API functionality
- database connectivity
- ETL execution
- cron automation
- analytics updates
- VPS accessibility

Orders placed through the frontend were successfully stored in the transactional database and later reflected in the analytics database after ETL execution.

Conclusion

The project successfully developed and deployed a functional full-stack e-commerce application on a VPS environment. The system integrated frontend development, backend APIs, PostgreSQL databases, ETL automation, and analytics reporting into one working platform. The implementation demonstrated how cloud deployment and database processing can be applied in real-world web applications while also providing automated analytics generation through ETL pipelines and cron job scheduling.